

# Software Requirements Specification

---

## Dungeon Adventure Game

---

**Version 2.0 — As-Built / Verified Against Source**

Prepared by Abdullah Temori, Tarik Atasoy, Daniella Birungi

University of Washington Tacoma

June 2026

---

## Table of Contents

---

- **1. Introduction**
  - 1.1 Purpose
  - 1.2 Document Conventions
  - 1.3 Intended Audience and Reading Suggestions
  - 1.4 Project Scope
  - 1.5 References
- **2. Overall Description**
  - 2.1 Product Perspective
  - 2.2 Product Features
  - 2.3 User Classes and Characteristics
  - 2.4 Operating Environment
  - 2.5 Design and Implementation Constraints
  - 2.6 User Documentation
  - 2.7 Assumptions and Dependencies
- **3. System Features**
  - 3.1 Dungeon Generation
  - 3.2 Hero Selection and Character System
  - 3.3 Movement and Navigation
  - 3.4 Combat System
  - 3.5 Item and Inventory System
  - 3.6 Difficulty Levels
  - 3.7 Save and Load System
  - 3.8 Audio System
  - 3.9 Win and Lose Conditions

- **4. External Interface Requirements**
  - 4.1 User Interfaces
  - 4.2 Hardware Interfaces
  - 4.3 Software Interfaces
  - 4.4 Communications Interfaces
- **5. Other Nonfunctional Requirements**
  - 5.1 Performance Requirements
  - 5.2 Safety Requirements
  - 5.3 Security Requirements
  - 5.4 Software Quality Attributes
- **6. Other Requirements**
  - 6.1 Database Requirements
  - 6.2 Internationalization and Localization
  - 6.3 Legal and Licensing
  - 6.4 Reuse Objectives
  - 6.5 Development Process Requirements
- **Appendices**
  - Appendix A: Glossary
  - Appendix B: Analysis Models
  - Appendix C: Issues List (Design-vs-Build Reconciliation)

---

## Revision History

---

| Name                                                     | Date       | Reason For Changes                                                                                                                                                                                                                                                                                    | Version |
|----------------------------------------------------------|------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------|
| Abdullah Temori                                          | 04/16/2026 | Initial draft                                                                                                                                                                                                                                                                                         | 1.0     |
| Daniella Birungi                                         | 04/20/2026 | Initial draft                                                                                                                                                                                                                                                                                         | 1.1     |
| Tarik Atasoy                                             | 04/22/2026 | Initial draft                                                                                                                                                                                                                                                                                         | 1.2     |
| Abdullah<br>Temori, Daniella<br>Birungi, Tarik<br>Atasoy | 06/08/2026 | <b>As-built revision: every requirement cross-referenced against the actual source tree ( <code>src/</code> ) and test suites ( <code>test/</code> ). Requirements that the code does not yet implement were corrected to match the stable system. Design-vs-build deltas captured in Appendix C.</b> | 2.0     |

---

## 1. Introduction

---

### 1.1 Purpose

This document provides a detailed description of the Dungeon Adventure Game software system. The purpose of this SRS is to define the functional and non-functional requirements for the game, including gameplay mechanics, system structure, and user interactions.

**Version 2.0 is an "as-built" specification.** Every requirement below has been verified line-by-line against the current source tree at `src/edu/uw/tcss/dungeoneer/` and the JUnit 5 suites at `test/edu/uw/tcss/dungeoneer/`. Where the original design (v1.x) described behavior that the stable codebase does not implement, the requirement has been rewritten to state the truth of the system, and the original intent is preserved in Appendix C so no information is lost.

## 1.2 Document Conventions

This document follows standard SRS formatting. Functional requirements are labeled `REQ-#`. Non-functional requirements use category prefixes (`PE-`, `SA-`, `SE-`, `AV-`, `RE-`, `US-`, `MA-`, `PO-`, `TE-`, `DB-`, `IN-`, `LE-`, `RU-`, `DP-`). Class, method, and file names are written in `monospace`. Any requirement whose behavior is **partially implemented or deferred** is marked **[Deferred]** or **[Partial]** and elaborated in Appendix C.

## 1.3 Intended Audience and Reading Suggestions

This document is intended for developers, team members, graders, and instructors. Readers should start with the Introduction and Overall Description, then proceed to System Features. Maintainers reconciling the documentation against the code should read Appendix C first.

## 1.4 Project Scope

The Dungeon Adventure Game is a Java desktop application that ships with **two interchangeable front ends** — a Java Swing GUI (default) and a text console view — both driven by a single shared model and controller. A player navigates a randomly generated dungeon, collects the four Pillars of OO, fights monsters in turn-based combat, and reaches the exit to win. The project demonstrates the MVC architectural pattern, the Observer pattern (via `PropertyChangeSupport`), the Factory pattern (`HeroFactory`, `MonsterFactory`), the Builder pattern (`DungeonBuilder`), the Singleton pattern (`AudioManager`), and Java object serialization for save/load.

## 1.5 References

- TCSS 360 Course Materials and Project Instructions ("Option 2 – Dungeon Adventure")
- Java SE 17 Documentation (Oracle)
- SQLite / Xerial `sqlite-jdbc` Documentation
- JUnit 5 (Jupiter) Documentation
- GitHub Documentation
- Team UML export: `project_docs_for_opus/UML_final.puml` / `UML_final.pdf`

---

## 2. Overall Description

---

## 2.1 Product Perspective

This is a standalone, single-player Java application built on the MVC architecture. The **model** package (`edu.uw.tcss.dungeoneer.model`) is free of any Swing dependency and contains all game state and rules. The **view** package (`edu.uw.tcss.dungeoneer.view`) provides two `GameView` implementations — `SwingView` and `ConsoleView` — that register as observers on `GameModel`. The **controller** package (`edu.uw.tcss.dungeoneer.controller`) contains `GameController`, which mediates all input. Persistence is provided by Java object serialization. Monster *names* may optionally be sourced from a SQLite database; when the database or its JDBC driver is unavailable, the system falls back to a built-in name list (see Section 4.3.1 and Appendix C, item C-5).

## 2.2 Product Features

- Random dungeon generation with a guaranteed-connected maze (recursive backtracking) plus a BFS traversability re-check.
- Hero selection from three classes, each with a unique special skill.
- Room-by-room movement with automatic room-entry effects.
- Turn-based combat with multi-attack rounds, blocking, and monster self-heal.
- Three collectible item types: Healing Potions, Vision Potions, and Bombs, plus Pits as hazards and the four Pillars of OO as win objectives.
- Save/load functionality using Java serialization (single default save file).
- Three difficulty levels (Easy, Medium, Hard) controlling **dungeon size** and **monster spawn density**.
- Background music and sound effects via the Java Sound API, with a mute toggle and graceful silent fallback when no audio device is present.
- A hidden cheat option that reveals the full dungeon map.

## 2.3 User Classes and Characteristics

Primary user: a single player with basic computer skills and no programming experience required. A secondary user class is the *grader/tester*, who uses the hidden cheat option and the console front end to inspect dungeon state.

## 2.4 Operating Environment

Runs on a Java Runtime Environment (Java 17 or newer) on Windows, macOS, or Linux. The default launch presents a Swing GUI; passing `--console` launches the text view.

## 2.5 Design and Implementation Constraints

Must use Java and the MVC architecture, apply the Factory and Builder patterns, follow OOP principles, and keep the `model` package free of Swing dependencies so it remains unit-testable and reusable. Random behavior in dungeon generation is driven by an injectable `java.util.Random` (`DungeonBuilder.random(Random)`). Combat, special-skill, and heal rolls currently use `Math.random()` internally (see TE-2 and Appendix C, item C-6).

## 2.6 User Documentation

- **In-game Help menu** (Swing): `Help > Instructions` opens a dialog describing controls, items, and objectives; `Help > About` shows team and course info.
- **README.md** — setup and launch instructions and Java version requirement.
- **Console hints** — the console front end prints a command legend each turn and a hero-class / difficulty guide on the new-game screens.

## 2.7 Assumptions and Dependencies

**Assumptions:** Java 17+ is installed; the local file system is writable for the save file; audio hardware is optional; only one game instance runs at a time.

**Dependencies (as present in the repository):**

- Java Swing ( `javax.swing` ) — GUI.
  - `javax.sound.sampled` — audio (bundled with the JRE).
  - Java object serialization — save/load (bundled with the JRE).
  - JUnit 5 (Jupiter 5.8.1 / 5.10.0 jars in `lib/` ) — testing only.
  - SQLite via `org.sqlite.JDBC` — **optional**. The Xerial `sqlite-jdbc` driver is **not** currently bundled in `lib/` , and no `monsters.db` is checked in, so the runtime currently uses the built-in fallback monster list (see Appendix C, item C-5). A helper, `Database Setup/DatabaseSetup` , can generate `monsters.db` when a driver is on the classpath.
  - Git/GitHub — version control; YouTrack — planning and hour logging.
- 

## 3. System Features

---

### 3.1 Dungeon Generation

#### 3.1.1 Description and Priority

A randomly generated 2D grid of `Room` objects, regenerated fresh for each new game. Generation lives in `DungeonBuilder` ; layout storage lives in `Dungeon` . Priority: High.

#### 3.1.2 Stimulus/Response Sequences

Player selects New Game → `DungeonBuilder.build()` carves a maze, marks the entrance/exit, places pillars, sprinkles items, and populates monsters → the layout is verified by `Dungeon.isTraversable()` → the hero is placed on the entrance.

#### 3.1.3 Functional Requirements

- **REQ-3.1.1:** The dungeon shall be a 2D grid of `Room` objects, each holding a `Map<Direction, Boolean>` of doors on the N, S, E, and W sides.

- **REQ-3.1.2:** Dungeon size shall be derived from difficulty: Easy 5×5, Medium 7×7, Hard 10×10 ( `Difficulty` enum). A custom size may be supplied for tests via `DungeonBuilder.setSize(rows, cols)`.
- **REQ-3.1.3:** The dungeon shall contain exactly one Entrance (top-left, `(0,0)`) and one Exit (bottom-right, `(rows-1, cols-1)`). Both rooms shall have all other contents cleared. The four Pillars of OO (Abstraction, Encapsulation, Inheritance, Polymorphism) shall be placed in four distinct rooms that are neither the entrance nor the exit.
- **REQ-3.1.4:** The maze shall be carved with iterative recursive-backtracking DFS, producing a spanning tree in which every room is reachable. After population, the builder shall verify connectivity with a BFS ( `Dungeon.isTraversable()` ) and regenerate if the check ever fails.
- **REQ-3.1.5:** Each normal room (not entrance, exit, or a pillar room) shall have an **independent 10%** chance of a Pit, a 10% chance of a Healing Potion, a 10% chance of a Vision Potion, and a 10% chance of a Bomb ( `DungeonBuilder.DEFAULT_ITEM_CHANCE = 0.10` ). Pit damage is rolled in the range **1–20** at placement time.
- **REQ-3.1.6:** Monsters shall be placed as follows: the entrance never receives a monster; every pillar room, the exit room, and every room **orthogonally adjacent** to the exit shall receive a strong guardian (an Ogre); all other rooms shall receive a monster with probability equal to the per-difficulty spawn chance (Easy 0.15, Medium 0.25, Hard 0.35) drawn from `MonsterFactory.createRandom()`.
- **REQ-3.1.7:** Dungeon construction shall use the Builder pattern ( `DungeonBuilder` ) with fluent setters ( `setDifficulty` , `setSize` , `itemChance` , `monsterChance` , `monsterSupplier` , `strongMonsterSupplier` , `random` ).

## 3.2 Hero Selection and Character System

### 3.2.1 Description and Priority

The player supplies a name and selects one of three hero classes, each with unique stats and a special skill, created through `HeroFactory` . Priority: High.

### 3.2.2 Stimulus/Response Sequences

Player enters a name → selects Warrior, Priestess, or Thief → `HeroFactory.createHero(type, name)` instantiates the hero → the game begins.

### 3.2.3 Functional Requirements

- **REQ-3.2.1:** The system shall offer three hero classes with the following exact constructor-defined stats:

| Class     | HP  | Atk Speed | Hit Chance | Damage | Block |
|-----------|-----|-----------|------------|--------|-------|
| Warrior   | 125 | 4         | 0.80       | 35–60  | 0.20  |
| Priestess | 75  | 5         | 0.70       | 25–45  | 0.30  |

| Class | HP | Atk Speed | Hit Chance | Damage | Block |
|-------|----|-----------|------------|--------|-------|
| Thief | 75 | 6         | 0.80       | 20–40  | 0.40  |

- **REQ-3.2.2:** Each hero class shall have a unique special skill:
- **Warrior — Crushing Blow:** 40% chance to deal **75–175** damage; otherwise the attempt fails ( `SPECIAL_FAIL` ).
- **Priestess — Heal:** always succeeds and restores **20–50** HP to herself ( `SPECIAL_HEAL` ).
- **Thief — Surprise Attack:** a single roll yields 40% success (two normal attacks this turn), 20% caught ( `SPECIAL_CAUGHT` , no attack), or 40% fallback (one normal attack).
- **REQ-3.2.3:** Hero creation shall be class- and name-driven only. Hero stats are **not** modified by difficulty in the current build (see Appendix C, item C-1). The hero begins with zero Healing Potions, zero Vision Potions, zero Bombs, and zero Pillars.
- **REQ-3.2.4:** A `HeroFactory` shall instantiate hero objects from a case-insensitive class name; an unknown class name shall raise `IllegalArgumentException` .

### 3.3 Movement and Navigation

#### 3.3.1 Description and Priority

The hero moves room-by-room via directional input; room effects trigger automatically on entry. Priority: High.

#### 3.3.2 Stimulus/Response Sequences

Player picks a direction → `Dungeon.moveHero(direction)` validates door + bounds → `GameController.onEnterRoom()` auto-collects items, applies pit damage, and starts combat if a monster is present.

#### 3.3.3 Functional Requirements

- **REQ-3.3.1:** Movement in a direction shall succeed only when the current room has an open door on that side and the target cell is within the grid; otherwise the view reports "There is no door to the ...".
- **REQ-3.3.2:** On entering a room with a Pit, the hero shall automatically lose the pit's stored damage (1–20 HP). HP is clamped at zero ( `DungeonCharacter.setHitPoints` uses `Math.max(0, ...)` ).
- **REQ-3.3.3:** On entering a room with a Healing Potion, Vision Potion, or Bomb, the item shall be auto-added to the hero's inventory and removed from the room ( `Room.pickUpItems` ). The view reports each pickup.
- **REQ-3.3.4:** On entering a room with a Pillar of OO, the Pillar shall be auto-collected into the hero's `Set<Pillar>` (duplicates impossible) and removed from the room.
- **REQ-3.3.5:** A hidden cheat option shall reveal the entire dungeon map: in the console it is the keyword `XYZZY` ; in the Swing GUI it is `Help > Cheat Mode` . While cheat mode is on, the full map is shown after every move.

## 3.4 Combat System

### 3.4.1 Description and Priority

Turn-based combat begins automatically when the hero enters a room containing a monster and ends when either side reaches 0 HP. Combat rules live in `Combat`; events are emitted as `CombatEvent` objects so the model performs no I/O. Priority: High.

### 3.4.2 Stimulus/Response Sequences

Hero enters a monster room → `GameModel.startCombat()` → each round the view calls `Combat.executeHeroAction(action)` with one of {Attack, Special Skill, Use Healing Potion, Use Bomb} → the monster retaliates → repeat until one side reaches 0 HP.

### 3.4.3 Functional Requirements

- **REQ-3.4.1:** Attack counts per round shall be computed as `attacksPerRound(faster, slower) = max(1, floor(faster / slower))`. The hero's attack count for an ATTACK action is `max(heroAttacks, monsterAttacks)`, ensuring a hero never gets fewer swings than the monster. The monster's retaliation count is `floor(monsterSpeed / heroSpeed)`, at least 1.
- **REQ-3.4.2:** Each swing shall resolve against the attacker's chance-to-hit; on a hit, damage is rolled uniformly in the attacker's `[minDamage, maxDamage]` and applied to the opponent (`DungeonCharacter.attack()`).
- **REQ-3.4.3:** Each incoming monster swing shall be subject to the hero's block roll (`Hero.block()` against `chanceToBlock`); a successful block yields an `ATTACK_BLOCKED` event and zero damage.
- **REQ-3.4.4:** After the hero deals damage and if the monster is still alive, the monster shall get one heal check per round (`Monster.heal()`), succeeding with its `chanceToHeal` and restoring a roll in `[minHeal, maxHeal]`, capped at the monster's maximum HP. A fainted monster cannot heal.
- **REQ-3.4.5:** Monster identities shall be produced by `MonsterFactory`. The factory reads the **monster name** from the SQLite `monsters` table when available and instantiates the matching concrete subclass; **the numeric statistics are defined in the `Ogre`, `Gremlin`, and `Skeleton` subclasses**, not read from the database (see Appendix C, item C-5).  
Monster stats are:

| Monster  | HP  | Atk Speed | Hit Chance | Damage | Heal Chance | Heal  |
|----------|-----|-----------|------------|--------|-------------|-------|
| Ogre     | 200 | 2         | 0.60       | 30–60  | 0.10        | 30–60 |
| Gremlin  | 70  | 5         | 0.80       | 15–30  | 0.40        | 20–40 |
| Skeleton | 100 | 3         | 0.80       | 30–50  | 0.30        | 30–50 |

- **REQ-3.4.6:** If the hero's HP reaches zero during combat, the game shall set the game-over flag and transition to the game-over flow.



- **REQ-3.4.7:** On defeating a monster, the room shall be cleared of that monster and control shall return to room navigation; a `COMBAT_END` event (amount 1 = hero won, 0 = hero lost) is emitted.

## 3.5 Item and Inventory System

### 3.5.1 Description and Priority

Three collectible items (Healing Potions, Vision Potions, Bombs) are auto-collected on room entry. Priority: High.

### 3.5.2 Stimulus/Response Sequences

Items are auto-collected on entry. Vision Potions are usable during navigation; Healing Potions are usable in navigation and combat; Bombs are usable in combat. Inventory counts decrement on use.

### 3.5.3 Functional Requirements

- **REQ-3.5.1:** Healing Potions shall restore **5–15** HP. The amount is rolled when the potion is consumed. They may be used in combat ( `HeroAction.USE_HEALING_POTION` ) or navigation ( `GameController.handleUseHealingPotion` ).
- **REQ-3.5.2:** Vision Potions shall reveal the contents of up to the 8 rooms surrounding the hero ( `Dungeon.getSurroundingRooms` ). Corner positions reveal 3, edge positions 5, and interior positions 8.
- **REQ-3.5.3:** Bombs shall deal **75–150** damage to the current monster in combat and are consumed on use regardless of outcome ( `Bomb` , `Hero.useBomb` ).
- **REQ-3.5.4:** Bombs shall spawn in normal rooms with the same independent 10% per room chance as the other items and shall never appear in the entrance, exit, or pillar rooms.
- **REQ-3.5.5:** The hero's inventory counts (Healing Potions, Vision Potions, Bombs) and the pillars collected shall be shown in the Swing Hero Status panel and the console status line.
- **REQ-3.5.6:** The hero may hold unlimited quantities of each item type (simple integer counters; pillars are a `Set` ).

## 3.6 Difficulty Levels

### 3.6.1 Description and Priority

Selected on the new-game screen. In the current build, difficulty affects **dungeon size** and **monster spawn density**. Priority: Medium (Extra Credit).

### 3.6.2 Stimulus/Response Sequences

Player selects New Game → picks Easy, Medium, or Hard → `DungeonBuilder` is configured via `setDifficulty()` and the chosen `Difficulty` is stored in `GameModel` .

### 3.6.3 Functional Requirements

- **REQ-3.6.1:** The system shall support three difficulty levels: `EASY`, `MEDIUM`, and `HARD` (`Difficulty` enum).
- **REQ-3.6.2:** Difficulty shall set the dungeon grid size — Easy 5×5, Medium 7×7, Hard 10×10.
- **REQ-3.6.3:** Difficulty shall set the default monster spawn chance for normal rooms — Easy 0.15, Medium 0.25, Hard 0.35 (`DungeonBuilder.DEFAULT_MONSTER_CHANCE`).
- **REQ-3.6.4: [Deferred]** Difficulty-based modifiers to hero HP, monster damage, item spawn rates, hero block chance, pit damage range, and starting inventory are **not implemented** in the current build. Item chance is a fixed 10% and pit damage is a fixed 1–20 across all difficulties (see Appendix C, item C-1).
- **REQ-3.6.5:** The selected difficulty shall be persisted in `GameModel`, serialized with the save file, and displayed in the Swing Hero Status panel.

## 3.7 Save and Load System

### 3.7.1 Description and Priority

The player can save and reload game state via the File menu (Swing) or the `SAVE / LOAD` commands (console), using Java serialization. Priority: High.

### 3.7.2 Stimulus/Response Sequences

File > Save Game → the entire `GameModel` object graph is serialized to a `.sav` file. File > Load Game → the graph is deserialized and the view is re-registered as a listener (listeners are transient).

### 3.7.3 Functional Requirements

- **REQ-3.7.1:** The system shall save and load using `ObjectOutputStream` / `ObjectInputStream` (`SaveLoadManager`).
- **REQ-3.7.2:** The serialized graph shall include the full `Dungeon` grid, the `Hero` (name, class, stats, inventory, pillars), the hero position (stored inside `Dungeon`), monster states, win/lose flags, and the `Difficulty`. The transient `PropertyChangeSupport` is rebuilt on load via `readObject` / `readResolve`.
- **REQ-3.7.3:** The system shall support a single default save slot, `dungeoneer_save.sav`, in the working directory. Both the Swing menu and the console use this default path (no file chooser in the current build; see Appendix C, item C-2).
- **REQ-3.7.4:** If a save file is missing or cannot be read/deserialized, `SaveLoadManager` shall return `null` and the controller shall display an error message and remain on the current screen without crashing.
- **REQ-3.7.5:** All serializable classes (`GameModel`, `Dungeon`, `Room`, `Hero` and subclasses, `Monster` and subclasses, `Combat`, `CombatEvent`, and item classes that implement `Serializable`) shall declare an explicit `serialVersionUID`.

## 3.8 Audio System

### 3.8.1 Description and Priority

Background music and sound effects via the Java Sound API, managed by the `AudioManager` singleton. Priority: Medium (Extra Credit).

### 3.8.2 Stimulus/Response Sequences

Menu music begins on Swing launch (`MUSIC_MENU`). Movement triggers a move SFX. Combat events can be mapped to SFX via `AudioManager.playCombatSFX(CombatEvent)`.

### 3.8.3 Functional Requirements

- **REQ-3.8.1:** Background music shall loop continuously using `Clip.LOOP_CONTINUOUSLY`. Menu music starts at launch. Dungeon and combat music tracks (`MUSIC_DUNGEON`, `MUSIC_COMBAT`) are bundled and supported by the API but are not auto-switched by the controller flow in the current build (see Appendix C, item C-3).
- **REQ-3.8.2:** Sound-effect constants shall exist for movement, item pickup, pit, attack hit, attack miss, bomb, hero death, monster defeat, and game win. Move SFX is wired to the Swing navigation buttons; combat SFX mapping is provided by `playCombatSFX`.
- **REQ-3.8.3:** Audio shall use `javax.sound.sampled` and PCM-encoded **WAV** files loaded from the classpath. The 12 WAV assets reside in `src/edu/uw/tcss/dungeoneer/view/sounds/` and are loaded via `getResourceAsStream("sounds/<file>.wav")`.
- **REQ-3.8.4:** If audio is unavailable (no mixer/sound card) or a clip fails to load, the system shall catch the exception, log a warning, and continue running silently — no crash. Availability is probed once in the constructor.
- **REQ-3.8.5:** A mute toggle shall be available at `Help > Mute Audio` in the Swing GUI (`AudioManager.toggleMute`); muting stops music and makes all play calls no-ops, unmuting resumes the looped track.

## 3.9 Win and Lose Conditions

### 3.9.1 Description and Priority

Win/lose states are tracked in `GameModel` and evaluated by `GameController.checkWinLose()`. Priority: High.

### 3.9.2 Functional Requirements

- **REQ-3.9.1:** The player wins by collecting all four pillars **and** standing in the Exit room. On a win, the controller sets the win and game-over flags and reveals the full dungeon map.
- **REQ-3.9.2:** The player loses if the hero's HP reaches 0. On a loss, the controller sets the game-over flag (`win = false`) and reveals the full dungeon map.

- **REQ-3.9.3:** A new game may be started at any time via `File > New Game` (Swing) or by returning to the welcome menu (console) after a game ends.
- 

## 4. External Interface Requirements

---

### 4.1 User Interfaces

The game provides two `GameView` implementations.

**Swing GUI ( `SwingView` ) — single `JFrame` (minimum 800×600):** - **Title bar** (NORTH): "× Dungeon Adventure ×". - **Hero Status panel** (WEST): hero name/class, HP, Healing Potions, Vision Potions, Bombs, Pillars, and Difficulty. - **Game Log** (CENTER): scrollable, read-only event history (room ASCII art, pickups, combat messages). - **Navigation / Combat card panel** (EAST, `CardLayout`): a 3×3 compass of N/S/E/W buttons plus Use Potion and Use Vision buttons; swaps to the `CombatPanel` during battles. - **Menu bar:** **File** (New Game, Save Game, Load Game, Exit) and **Help** (Instructions, About, Mute Audio, Cheat Mode). New Game is collected via dialog prompts; there are no separate full-screen main-menu / difficulty / end screens in the current build (see Appendix C, item C-4).

**Console view ( `ConsoleView` ):** text rendering of rooms, the dungeon map, vision results, combat, and hero stats; driven by the menu/loop in `DungeonAdventure`.

- **REQ-4.1.1:** Navigation buttons shall reflect available doors; buttons are enabled/updated to communicate valid moves after each room change.
- **REQ-4.1.2:** The Swing GUI shall support keyboard input via `InputMap` / `ActionMap` bindings (e.g. W/A/S/D and arrow keys for movement) in addition to mouse.
- **REQ-4.1.3:** The GUI shall enforce a minimum window size of 800×600 pixels.

### 4.2 Hardware Interfaces

The game is a standalone desktop application requiring only commodity hardware.

- Display capable of at least 800×600; the Swing layout is resizable.
- Keyboard for name entry, menu navigation, and shortcuts.
- Pointing device for menu/button/dialog interaction.
- Audio output is optional (graceful silent fallback per REQ-3.8.4).
- Local storage for the save file.
- **REQ-4.2.1:** The system shall not require hardware beyond keyboard, pointing device, display, and local file system.
- **REQ-4.2.2:** The system shall run without crashing when no audio device is present.

### 4.3 Software Interfaces

| Component                                  | Version                                        | Source            | Purpose                                                 |
|--------------------------------------------|------------------------------------------------|-------------------|---------------------------------------------------------|
| Java SE Runtime                            | 17 LTS or newer                                | Oracle / Adoptium | Execution environment                                   |
| SQLite JDBC driver                         | org.xerial:sqlite-jdbc (optional; not bundled) | Maven Central     | Read monster <i>names</i> from <code>monsters.db</code> |
| <code>javax.sound.sampled</code>           | Bundled with JRE                               | Java Platform     | WAV playback                                            |
| Java Object Serialization                  | Bundled with JRE                               | Java Platform     | Save/load                                               |
| Java Swing<br>( <code>javax.swing</code> ) | Bundled with JRE                               | Java Platform     | GUI                                                     |
| JUnit 5 (Jupiter)                          | 5.8.1 / 5.10.0 (in <code>lib/</code> )         | Maven Central     | Unit testing (dev only)                                 |

#### 4.3.1 SQLite Database ( `monsters.db` )

- **Data out:** the `name` column from the `monsters` table.
- **Connection:** `jdbc:sqlite:database/monsters.db` via the Xerial driver, opened by `MonsterFactory` at construction.
- **REQ-4.3.1:** When the driver and database are present, the factory shall read all monster names once at construction and cache them. **The numeric monster statistics are taken from the concrete subclasses, not the database.**
- **REQ-4.3.2:** If the driver is absent, the file is missing, or a query fails, the factory shall log a warning and fall back to the built-in name list `{Ogre, Gremlin, Skeleton}` without crashing. (In the current repository state this fallback path is the active one — see Appendix C, item C-5.)
- **REQ-4.3.3:** All parameterized lookups ( `createByName` ) shall use `PreparedStatement` ; the one-time name load uses a constant, input-free SQL string.

#### 4.3.2 Java Object Serialization (Save/Load)

- **Data in/out:** the `GameModel` object graph ( `Dungeon` , `Hero` , hero position, monster states, flags, `Difficulty` ).
- **REQ-4.3.4:** Save/load shall use `ObjectOutputStream` / `ObjectInputStream` .
- **REQ-4.3.5:** All persisted classes shall declare an explicit `serialVersionUID` .

#### 4.3.3 Java Sound ( `javax.sound.sampled` )

- **Data in:** WAV files bundled under `view/sounds/` and loaded from the classpath.
- **REQ-4.3.6:** All audio resources shall be PCM-encoded WAV files so no third-party codec is required.

#### 4.3.4 File System

- `./database/monsters.db` — optional, read-only monster names (path used by `MonsterFactory`).
- `./dungeoneer_save.sav` — the default save file, created at runtime.
- Classpath `sounds/*.wav` — bundled audio assets.
- **REQ-4.3.7:** The save operation shall write the save file in the working directory; a missing file on load is reported, not fatal.

## 4.4 Communications Interfaces

The game is a single-player, offline desktop application.

- **REQ-4.4.1:** The system shall not open network sockets or make network requests.
- **REQ-4.4.2:** No external communication protocols are required or supported.

---

## 5. Other Nonfunctional Requirements

### 5.1 Performance Requirements

- **PE-1:** Dungeon generation (carve + populate + BFS check, with rebuild loop) shall complete in under 500 ms for all difficulties on a reference machine (dual-core 2.0 GHz, 4 GB RAM).
- **PE-2:** Movement and UI actions shall produce a visible response within 100 ms.
- **PE-3:** Combat rounds shall resolve within 250 ms so the log stays readable.
- **PE-4:** Save and load shall complete within 2 seconds for any game state.
- **PE-5:** Application start-up (JVM launch → main window/menu) shall complete within 5 seconds.
- **PE-6:** Resident memory shall not exceed 256 MB during Hard-difficulty play.

### 5.2 Safety Requirements

- **SA-1: [Deferred]** Overwrite-confirmation before replacing the save file is not implemented; the current build saves to the default path directly (see Appendix C, item C-2).
- **SA-2:** Save deletion is exposed only as a test helper ( `SaveLoadManager.deleteSave` ) and is not reachable from normal gameplay.
- **SA-3:** A failed or partial save shall not crash the game; a corrupt or missing save returns `null` on load and is reported via the view (REQ-3.7.4).

### 5.3 Security Requirements

- **SE-1:** The system shall read and write only within the working directory ( `./dungeoneer_save.sav` , `./database/monsters.db` , classpath `sounds/` ).
- **SE-2: [Deferred]** Deserialization currently uses standard Java `ObjectInputStream` without a class whitelist/ `ObjectInputFilter` . Only trusted, locally-created save files

should be loaded (see Appendix C, item C-7).

- **SE-3:** SQL lookups that take a parameter shall use `PreparedStatement` ; no user-supplied string is concatenated into SQL.
- **SE-4:** The application shall not issue `INSERT / UPDATE / DELETE` against `monsters.db` at runtime (the optional `DatabaseSetup` helper is a separate, developer-run utility, not part of gameplay).

## 5.4 Software Quality Attributes

### 5.4.1 Availability

- **AV-1:** The game shall be available whenever the host and JRE are available; no authentication, license check, or network dependency gates start-up.

### 5.4.2 Reliability

- **RE-1:** The application shall not crash during a normal play session in at least 95% of test runs.
- **RE-2:** Recoverable errors (missing audio device, missing monster database, corrupt save) shall be handled gracefully and reported, not fatal.

### 5.4.3 Usability

- **US-1:** A first-time player shall be able to start a new game, choose a hero, and make a first move within three minutes, aided by the Help menu / console prompts.
- **US-2:** Interactive controls shall have visible labels in English.
- **US-3:** Hero class, HP, inventory counts, pillars, and difficulty shall be visible during play (Swing Hero Status panel / console status line).

### 5.4.4 Maintainability

- **MA-1:** The source shall follow MVC: `model/` has no dependency on `view/` or `controller/` ; the view talks to the model only through `GameView` and `PropertyChange` events.
- **MA-2:** JUnit 5 tests shall cover, at minimum, dungeon generation, room pickup behavior, combat resolution, hero special skills, items, save/load, win/lose conditions, and controller flow. The repository currently contains **20 test classes with roughly 296 @Test methods** plus a `StubView` test double.
- **MA-3:** Public classes and methods in `model` shall carry Javadoc.

### 5.4.5 Portability

- **PO-1:** The application shall run without source changes on Windows 10+, macOS 12+, and Ubuntu 22.04 LTS (or equivalent) with a compatible JRE.
- **PO-2:** Room/dungeon rendering shall use `System.LineSeparator()` for newlines to stay platform-neutral.



### 5.4.6 Testability

- **TE-1:** The `model` package shall be free of Swing dependencies so it can be unit-tested without a GUI (verified: `model/` imports no `javax.swing`).
  - **TE-2: [Partial]** Dungeon generation accepts an injectable `Random` (`DungeonBuilder.random`). Combat, special-skill, heal, and item rolls currently use `Math.random()` internally and are therefore not seedable; see Appendix C, item C-6.
- 

## 6. Other Requirements

---

### 6.1 Database Requirements

- **DB-1:** The monster schema and seed data shall be reproducible from the `Database Setup/DatabaseSetup` helper, which creates a `monsters` table with columns `name`, `hp`, `min_damage`, `max_damage`, `speed`, `hit_chance`, `heal_chance`, `min_heal`, `max_heal` and seeds Ogre/Gremlin/Skeleton rows.
- **DB-2:** The seed data shall match the statistics in the Project Instructions and the concrete monster subclasses (Section 3.4.3).
- **DB-3: [Open]** The helper writes `monsters.db` to the working directory while `MonsterFactory` reads `database/monsters.db`; these paths shall be reconciled, and the Xerial driver shall be added to `lib/`, before the database path becomes the active source (Appendix C, item C-5).

### 6.2 Internationalization and Localization

- **IN-1:** Release 2.0 ships in English only. No localization framework is required.

### 6.3 Legal and Licensing

- **LE-1:** All third-party libraries shall be redistributable under licenses compatible with academic use (Apache 2.0, BSD, MIT, or equivalent).
- **LE-2:** Custom assets (audio) shall be team-created or permissively licensed, with attributions listed in `README.md`.

### 6.4 Reuse Objectives

- **RU-1:** The `model` package shall remain independent of the Swing view so a future release could substitute a JavaFX, web, or command-line front end without modifying model code (already demonstrated by the coexisting `ConsoleView` and `SwingView`).

### 6.5 Development Process Requirements

- **DP-1:** All source shall be managed in a single GitHub repository; members work on feature branches and merge via pull request.



- **DP-2:** Each member shall log development hours in YouTrack against tasks linked to requirements in this SRS.
- **DP-3:** Every functional pull request shall include or update at least one JUnit 5 test unless purely documentation/cosmetic.

## Appendix A: Glossary

| Term                  | Definition                                                                                                              |
|-----------------------|-------------------------------------------------------------------------------------------------------------------------|
| Adventurer / Hero     | The player-controlled character. Concrete classes: <code>Warrior</code> , <code>Priestess</code> , <code>Thief</code> . |
| Attack Speed          | Integer stat; the faster character gets <code>floor(faster/slower)</code> swings per round (min 1).                     |
| Block Chance          | Probability in [0,1] that a Hero negates an incoming swing.                                                             |
| Bomb                  | Single-use combat item dealing 75–150 damage to one monster.                                                            |
| Chance to Hit         | Probability in [0,1] that a swing lands; a miss deals 0 damage.                                                         |
| Cheat Option          | Hidden reveal of the full map: <code>XYZZY</code> (console) / <code>Help &gt; Cheat Mode</code> (Swing).                |
| Combat                | Turn-based sub-game entered when a Hero enters a room with a Monster.                                                   |
| CombatEvent           | Immutable value object describing one thing that happened in combat.                                                    |
| Difficulty            | <code>EASY</code> , <code>MEDIUM</code> , or <code>HARD</code> ; controls grid size and monster spawn density.          |
| Dungeon               | A 2D grid of <code>Room</code> objects forming the maze; also tracks hero position.                                     |
| DungeonBuilder        | Builder-pattern class that carves and populates a <code>Dungeon</code> .                                                |
| Entrance / Exit       | Unique rooms at <code>(0,0)</code> and <code>(rows-1, cols-1)</code> ; both otherwise empty.                            |
| Healing Potion        | Item that restores 5–15 HP.                                                                                             |
| HeroFactory           | Factory that creates a Hero from a class-name string.                                                                   |
| HP (Hit Points)       | A character's current health; 0 HP = defeated (clamped non-negative).                                                   |
| Monster               | Computer-controlled enemy: <code>Ogre</code> , <code>Gremlin</code> , <code>Skeleton</code> .                           |
| MonsterFactory        | Factory that creates monsters; reads names from SQLite when available, else uses a fallback list.                       |
| MVC                   | Model-View-Controller architecture used throughout.                                                                     |
| Pillar of OO          | One of four collectibles (Abstraction, Encapsulation, Inheritance, Polymorphism); all four + exit = win.                |
| Pit                   | Room hazard dealing 1–20 HP on entry.                                                                                   |
| PropertyChangeSupport | Observer mechanism by which <code>GameModel</code> notifies views.                                                      |
| Vision Potion         | Item revealing up to the 8 rooms surrounding the hero.                                                                  |
| WAV                   | Waveform Audio File Format, the only audio format used.                                                                 |

| Term     | Definition                                               |
|----------|----------------------------------------------------------|
| YouTrack | Team project-management tool for tasks and hour logging. |

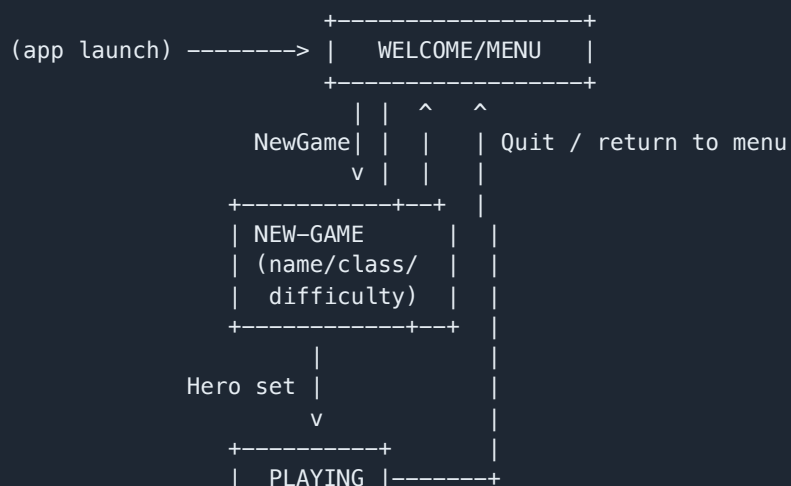
## Appendix B: Analysis Models

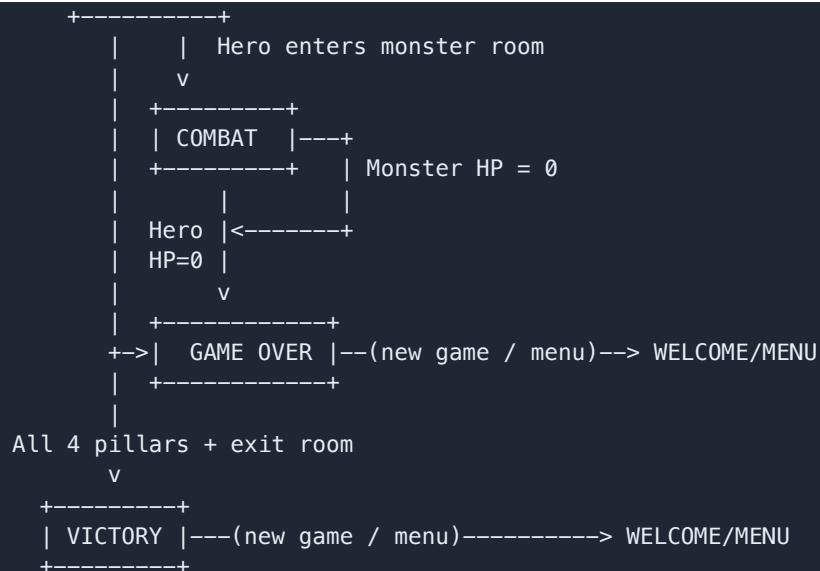
### B.1 Class Diagram (UML)

The full UML class diagram is maintained in `project_docs_for_opus/UML_final.puml` (and exported as `UML_final.pdf` / `.png` / `.svg`). Main structural elements:

- **MVC split:** `DungeonAdventure` (entry point) builds a `GameController` and a `GameView` (`SwingView` by default, `ConsoleView` with `--console`). `GameController` owns the `GameModel`. Views register on `GameModel` via `addPropertyChangeListener`.
- **Observer pattern:** `GameModel` uses `PropertyChangeSupport` and fires `gameOver`, `playerWon`, `dungeon`, `hero`, and `combat` property events.
- **Character hierarchy:** abstract `DungeonCharacter` → abstract `Hero` / abstract `Monster`; concrete leaves `Warrior`, `Priestess`, `Thief`, `Ogre`, `Gremlin`, `Skeleton`.
- **World model:** `Dungeon` composes a `Room[][]`; each `Room` may hold a `HealingPotion`, `VisionPotion`, `Bomb`, `Pillar`, a `Monster`, and a pit. Doors are an `EnumMap<Direction, Boolean>`.
- **Patterns in use:** Builder (`DungeonBuilder`), Factory (`HeroFactory`, `MonsterFactory`), Observer (`GameModel` + `GameView`), Singleton (`AudioManager`), and Serialization (`Serializable` on `GameModel`, `Dungeon`, `Room`, characters, `Combat`, `CombatEvent`, items).
- **Support classes:** `Combat` encapsulates turn resolution (REQ-3.4.x); `CombatEvent` carries combat results; `SaveLoadManager` wraps serialization; `AudioManager` wraps `javax.sound.sampled`; `MonsterPlacer` is a deprecated forwarder to `DungeonBuilder.placeMonsters`.

### B.2 Game-State Transition Diagram





Transitions not shown: Save/Load can fire from PLAYING without changing game state; the cheat option fires from PLAYING and returns to PLAYING.

### B.3 Combat-Round Flow (as implemented in `Combat.executeHeroAction`)

```

begin round (hero chose: ATTACK | SPECIAL_SKILL | USE_HEALING_POTION | USE_BOMB)
| resolve hero action:
|   ATTACK      -> swings = max(heroAtks, monsterAtks); each swing rolls hit/damage
|   SPECIAL_SKILL -> hero subclass skill (Crushing Blow / Heal / Surprise Attack)
|   USE_HEALING_POTION -> +5..15 HP if a potion is available, else ITEM_UNAVAILABLE
|   USE_BOMB      -> 75..150 damage if a bomb is available, else ITEM_UNAVAILABLE
|
| if monster took damage AND monster.hp > 0:
|   monster heal check (chanceToHeal -> +minHeal..maxHeal, capped at maxHP)
|
| if monster.hp <= 0 -> COMBAT_END (hero won); return
|
| monster retaliation: swings = floor(monsterSpeed / heroSpeed), min 1
|   for each swing while hero alive:
|     if hero.block() -> ATTACK_BLOCKED
|     else             -> monster.attack(hero) (hit/damage roll)
|
| if hero.hp <= 0 -> COMBAT_END (hero lost)

```

### B.4 Entity Summary

| Entity    | Key Fields                                                                                      | Referenced Requirements    |
|-----------|-------------------------------------------------------------------------------------------------|----------------------------|
| GameModel | dungeon, hero, difficulty, gameOver, playerWon, activeCombat, PropertyChangeSupport (transient) | REQ-3.7.x, REQ-3.9.x       |
| Dungeon   | rooms grid, rows, cols, heroRow, heroCol                                                        | REQ-3.1.x, REQ-3.3.x       |
| Room      | row, col, doors, pit+damage, healingPotion, visionPotion, bomb, pillar, monster                 | REQ-3.1.5, REQ-3.3.2–3.3.4 |

| Entity                             | Key Fields                                                                                | Referenced Requirements         |
|------------------------------------|-------------------------------------------------------------------------------------------|---------------------------------|
| Hero<br>(Warrior/Priestess/Thief)  | name, hp, dmg range, speed, hitChance, blockChance, potions, visions, bombs, pillarsFound | REQ-3.2.x, REQ-3.5.x            |
| Monster<br>(Ogre/Gremlin/Skeleton) | name, hp, maxHp, dmg range, speed, hitChance, healChance, heal range                      | REQ-3.4.4, REQ-3.4.5            |
| Combat                             | hero, monster, log, combatOver, heroWon                                                   | REQ-3.4.x                       |
| CombatEvent                        | type, actor, target, amount                                                               | REQ-3.4.x                       |
| AudioManager (singleton)           | musicClip, muted, audioAvailable                                                          | REQ-3.8.x                       |
| MonsterFactory                     | monsterNames, dbAvailable                                                                 | REQ-3.4.5, REQ-4.3.1, REQ-4.3.2 |
| HeroFactory                        | (stateless)                                                                               | REQ-3.2.4                       |
| DungeonBuilder                     | difficulty, item chance, monster chance, suppliers, random                                | REQ-3.1.x, REQ-3.6.x            |
| SaveLoadManager                    | (stateless utility)                                                                       | REQ-3.7.x                       |

## Appendix C: Issues List (Design-vs-Build Reconciliation)

The following items reconcile the original v1.x design intent with the verified state of the stable codebase. They are the **only** places where the prior SRS described behavior the current build does not (yet) fully implement; each is reflected in the corrected requirements above.

| #   | Issue                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                         |
|-----|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| C-1 | <b>Difficulty modifiers beyond size/spawn-rate are not implemented.</b> <code>Difficulty</code> carries only grid dimensions; <code>GameController.startNewGame</code> builds the dungeon with <code>setDifficulty()</code> alone. There is no hero HP bonus, monster-damage scaling, item-rate change, block-chance change, pit-damage change, or starting inventory by difficulty. Item chance is a constant 10% and pit damage a constant 1–20 across all levels. (Corrects v1.x REQ-3.2.3, 3.6.2, 3.6.4.) |
| C-2 | <b>Single default save slot; no file chooser; no overwrite confirmation.</b> Both Swing and console save/load use <code>SaveLoadManager.DEFAULT_SAVE_PATH</code> ( <code>dungeoneer_save.sav</code> ). Multiple named slots and a pre-overwrite confirmation dialog (v1.x REQ-3.7.3, SA-1) are deferred.                                                                                                                                                                                                      |
| C-3 | <b>Music track transitions are not auto-wired.</b> Only <code>MUSIC_MENU</code> is started (at Swing launch). <code>MUSIC_DUNGEON</code> and <code>MUSIC_COMBAT</code> assets and constants exist, and <code>playCombatSFX</code> maps events to SFX, but the controller does not switch background tracks on entering navigation/combat.                                                                                                                                                                     |
| C-4 | <b>GUI is a single dialog-driven window, not separate full screens.</b> New Game is gathered via <code>JOptionPane</code> prompts; there are no distinct main-menu, difficulty, or end-of-game screens. The mute and cheat toggles live under the <b>Help</b> menu — there is no separate <b>Settings</b> menu.                                                                                                                                                                                               |
| C-5 | <b>SQLite supplies monster <i>names</i> only, and is currently inactive at runtime.</b> <code>MonsterFactory</code> reads the <code>name</code> column from <code>database/monsters.db</code> ; all numeric stats come from the <code>Ogre / Gremlin / Skeleton</code> subclasses. No <code>monsters.db</code> is checked in and the Xerial <code>sqlite-</code>                                                                                                                                              |

| #   | Issue                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                       |
|-----|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
|     | <code>jdbc</code> driver is <b>not</b> in <code>lib/</code> , so <code>Class.forName("org.sqlite.JDBC")</code> fails and the built-in fallback name list is used. The <code>Database Setup/DatabaseSetup</code> helper writes <code>monsters.db</code> to the working directory (path mismatch with the factory's <code>database/monsters.db</code> ) and uses column names ( <code>hp</code> , <code>speed</code> , <code>hit_chance</code> , <code>heal_chance</code> ) that differ from the v1.x schema ( <code>hit_points</code> , <code>attack_speed</code> , <code>chance_to_hit</code> , <code>chance_to_heal</code> , plus a <code>type</code> column that the build does not use). |
| C-6 | <b>Combat randomness is not seedable.</b> <code>DungeonCharacter.attack</code> , <code>Hero.block</code> , <code>Monster.heal</code> , and the hero special skills use <code>Math.random()</code> directly. Only <code>DungeonBuilder</code> accepts an injectable <code>Random</code> . Full determinism for combat tests (v1.x TE-2) requires refactoring to an injected <code>Random</code> .                                                                                                                                                                                                                                                                                            |
| C-7 | <b>Deserialization is unfiltered.</b> <code>SaveLoadManager.loadGame</code> uses a plain <code>ObjectInputStream</code> with no <code>ObjectInputFilter</code> /class whitelist (v1.x SE-2/SE-3). Only locally-created, trusted save files should be loaded.                                                                                                                                                                                                                                                                                                                                                                                                                                |
| C-8 | <b>Priestess heal range is 20–50 (not 20–40).</b> The stable <code>Priestess</code> implementation heals <code>HEAL_MIN=20 .. HEAL_MAX=50</code> ; REQ-3.2.2 above reflects the code.                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                       |
| C-9 | <b>Audio assets live under the view package, not <code>resources/audio/</code>.</b> The 12 WAV files are in <code>src/edu/uw/tcss/dungeoneer/view/sounds/</code> and are loaded from the classpath via <code>getResourceAsStream("sounds/...")</code> .                                                                                                                                                                                                                                                                                                                                                                                                                                     |

*End of SRS. This document forms SRS v2.0 (as-built) of the Dungeon Adventure Game and supersedes v1.x. Every requirement herein has been verified against the source tree at `src/edu/uw/tcss/dungeoneer/` and the test suites at `test/edu/uw/tcss/dungeoneer/`.*